anything. If we assume `m≥2`, in fact, this is only possible when `node` is tree's root node (but still possible).
#3 Perform the merge, creating a new node.
#4/#5 Delete the two former children from current node.
#6 Add the result of the call to merge to the list of this node's children.
#7 Auxiliary function merge takes two nodes and returns a node that has the entries of both inputs.
#8 Verify that both nodes are leaves, or neither is.
#9 If nodes are leaves, return a new node whose points are the union of the nodes' sets of points.
#10 If nodes are internal, create a new node with children from both inputs.

This was the last piece of pseudocode we were missing for `delete`; before wrapping up this section, though, I'd like to exhort the reader to take another look at figure 10.19: the final result is a valid Ss-tree that doesn't violate any of its invariants but… let's be honest, the result is not that great, right? Now we have one huge sphere, `S3`, that is taking up almost all the space in its parent's bounding envelope, and significantly overlapping not just its sibling, but also its parent's other branch.

If you remember, this was mentioned as a risk of handling merges in case 2.b in the description for `delete`: it is, unfortunately a common side-effect of both node merges and moving nodes/points among siblings; especially when the choice of the entry to move is constrained, a far entry can be picked up for merge/move, and – as in the example in figure 10.19 – in the long run, after many deletions, this can make the tree unbalanced.